

Teste Baseado em Erros (*Teste de Mutação*)

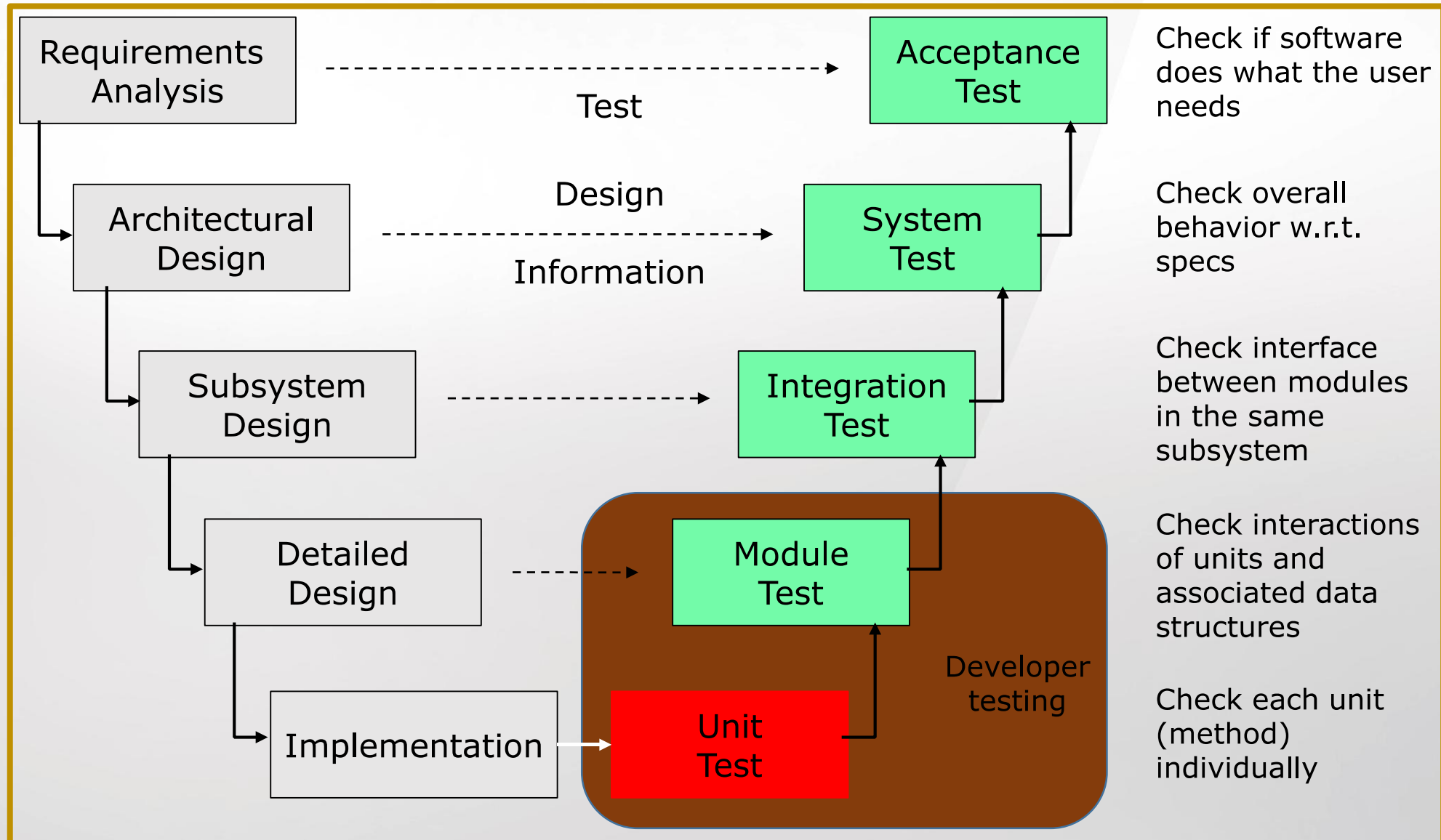
Prof. Iaçanã Ianiski Weber



Delamaro, M.E.; Maldonado J.C.; Jino, M. ***Introdução ao Teste de Software***. Elsevier. 2007.

✓ <https://www.amazon.com.br/Introdu%C3%A7%C3%A3o-Software-M%C3%A1rcio-Eduardo-Delamaro/dp/8535226346>

- Capítulo 5



- ✓ Técnicas:
 - Funcional (Capítulo 2)
 - Estrutural (Capítulo 4)
 - Baseada em Erros (Capítulo 5)
- ✓ A diferença está na fonte utilizada para *definir os requisitos de teste!!*
- ✓ Mas o **objetivo** das técnicas é similar:
 - Conduzir o processo de teste de uma forma mais sistemática
 - Escolher casos de teste que representem o domínio

Conceito:

- ✓ Também conhecido como **Teste de Mutação**
- ✓ É uma forma de avaliar a qualidade de um conjunto de casos de teste
- ✓ Baseia-se na análise do código fonte
 - Logo, também é do tipo de *Teste de Caixa Branca*

Hipóteses:

- ✓ **Hipótese do programador competente**
 - A maioria dos erros cometidos em programas são relativos a detalhes (operador errado, confusão com nome de variáveis, etc.)
- ✓ **Hipótese do acoplamento**
 - Um conjunto de casos de teste capaz de detectar erros relativos a detalhes será capaz de detectar erros mais complexos

Ideia básica:

- ✓ A partir do programa original geram-se programas com pequenas falhas e criam-se casos de teste capazes de detectá-las
- ✓ Para verificar se um teste detecta a existência de falhas introduzidas, compara-se o comportamento do mutante com o do programa original

Definições:

- ✓ **Mutante:** dado um programa P , um mutante de P é um programa P' sintaticamente correto obtido a partir de uma alteração no código fonte definida por um operador pré-estabelecido chamado *operador de mutação*
- ✓ **Morte de mutantes:** Um teste T mata um mutante P' se o resultado da execução de P' para T é diferente do resultado da execução de P para T

Processo de teste de mutação:

1. Iniciar com o programa **P** a ser testado e um conjunto de testes **T** (possivelmente vazio)
2. Gerar um conjunto **M** de mutantes de **P**, através de operadores de mutação (pequenas alterações sintaticamente corretas) no programa original
3. Verificar se os testes em **T** matam todos os mutantes em **M**
4. Se todos os mutantes foram mortos, o conjunto **T** é adequado, e o processo é encerrado
5. Senão, acrescenta-se um teste para cada mutante **M** sobrevivente, até que o conjunto **T** seja adequado

Exemplos de operadores de mutação:

- ✓ Substituição de uma variável por outra
- ✓ Substituição de uma variável escalar por um vetor
- ✓ Substituição de um vetor por uma variável escalar
- ✓ Substituição de operador relacional
- ✓ Substituição de operador aritmético
- ✓ Etc.

EXEMPLO

Enunciado:

- ✓ A função *max* recebe como argumento um ponteiro para uma lista de valores e o tamanho dessa lista, e retorna a posição na lista do maior valor encontrado. Se o maior valor estiver em mais de uma posição, a função deve retornar a 1ª posição encontrada.

```
int max (int* input, int size) {
    int j, result;
    result = 0;
    for (j = 1; j < size; j++) {
        if (input[j] > input[result])
            result = j;
    }
    return result;
}
```

teste	input[0]	input[1]	input[2]	size	Resultado Esperado	Resultado Obtido
1	1	2	3	3	2	2
2	3	1	2	3	0	0
3	1	3	2	3	1	1

Enunciado:

- ✓ A função *max* recebe como argumento um ponteiro para uma lista de valores e o tamanho dessa lista, e retorna a posição na lista do maior valor encontrado. Se o maior valor estiver em mais de uma posição, a função deve retornar a 1ª posição encontrada.

```
int max (int* input, int size) {
    int j, result;
    result = 0;
    for (j = 1; j < size; j++) {
        if (input[j] >= input[result])
            result = j;
    }
    return result;
}
```

Conjunto Inicial:

teste	input[0]	input[1]	input[2]	size	Resultado Esperado	Resultado Obtido
1	1	2	3	3	2	2
2	3	1	2	3	0	0
3	1	3	2	3	1	1

Conjunto que mata o Mutante:

teste	input[0]	input[1]	input[2]	size	Resultado Esperado	Resultado Obtido
1	1	2	3	3	2	2
2	3	1	2	3	0	0
3	1	3	2	3	1	1
4	1	2	2	3	1	2

```

1 void main()
2 {
3   int x, y, pow;
4   float z, ans;
5
6   scanf("%d %d", &x, &y);
7   if (y >= 0)
8     pow = y;
9   else
10    pow = -y;
11   z = 1;
12   while (pow-- > 0)
13     z = z*x;
14   if (y < 0)
15     z = 1 / z;
16   ans = z + 1;
17   printf("%-5.2f", ans);
18 }
```

> *Mutações sugeridas – remoções*

1	scanf("%d %d", &x, &y)
2	if (y >= 0) pow = y; else pow = -y;
3	pow = y
4	pow = -y
5	z = 1
6	while (pow-- > 0) z = z * x;
7	z = z * x
8	if (y < 0) z = 1 / z
9	z = 1 / z
10	ans = z + 1
11	printf("%-5.2f", ans)

```

1 void main()
2 {
3     int x, y, pow;
4     float z, ans;
5
6     scanf("%d %d", &x, &y);
7     if (y >= 0)
8         pow = y;
9     else
10        pow = -y;
11    z = 1;
12    while (pow-- > 0)
13        z = z*x;
14    if (y < 0)
15        z = 1 / z;
16    ans = z + 1;
17    printf("%-5.2f", ans);
18 }
```

> Mutações sugeridas – **mudança do operador relacional**

```

12 if (y > 0)
13 if (y < 0)
14 if (y == 0)
15 if (y <= 0)
16 if (y != 0)
17 while (pow-- < 0)
18 while (pow-- == 0)
19 while (pow-- >= 0)
20 while (pow-- <= 0)
21 while (pow-- != 0)
22 if (y > 0)
23 if (y == 0)
24 if (y >= 0)
25 if (y <= 0)
26 if (y != 0)
```